

SurgeonSite

Full Project Breakdown

An in-depth guide to the product model, source tree, frontend and backend wiring, relational data design, operational setup, test coverage, and current implementation gaps.

79 tracked files	8,761 tracked text lines	41 JSON API routes	28 database tables
29 HTML pages	27 ORM model classes	5 Alembic migrations	17/17 tests passing

Repository: /home/mj/Desktop/projects/SurgeonSite

Reviewed revision: 7263815 on branch admin-controls (also at main and origin/main)

Audit date: September 20, 2026

Scope: Entire tracked project plus runtime configuration shape. Private .env values were deliberately excluded.

Contents

1. Executive understanding
2. System architecture
3. Repository and file map
4. Frontend architecture and screens
5. Backend module breakdown
6. Complete API catalog
7. Database model and relationships
8. End-to-end workflows
9. Security and privacy model
10. Configuration, containers, deployment, backup
11. Automated tests and validation
12. Implemented, partial, and planned behavior
13. Findings and prioritized risks
14. Recommended next work
15. Appendices

1. Executive understanding

SurgeonSite is a compact, single-service web application for a community-maintained directory of gender-affirming surgeons. The public-facing brand in the HTML is **OpenSurgery**, while backend configuration currently names the service **transdoc.wiki**. Its distinguishing product idea is to attach several different kinds of information to one stable surgeon identity without mixing them together:

- **Factual profile content** is revisioned and source-oriented.
- **Patient reviews** are structured first-person records.
- **Talk discussions** hold editorial debate.
- **Edit proposals** wait for trusted review before becoming published history.
- **Practices, procedures, techniques, media, messages, reports, and audit events** remain separate relational records linked through foreign keys.

The implementation uses FastAPI and SQLAlchemy on the server, PostgreSQL in normal deployment, Alembic for schema evolution, and plain HTML/CSS/JavaScript in the browser. FastAPI serves both the browser assets and the JSON API, so there is no separate frontend build tool or Node.js service. The browser hydrates mostly static page shells by calling `/api/v1`.

Current maturity: This is a functional prototype with real persistence for accounts, settings, surgeon submissions, proposals, reviews, talk topics, messages, password resets, uploads, and several administrator actions. It is not launch-ready. Review/media moderation, block enforcement, general notification delivery, production storage, rate limiting, legal copy, and several UI-to-model connections remain incomplete.

What is working today

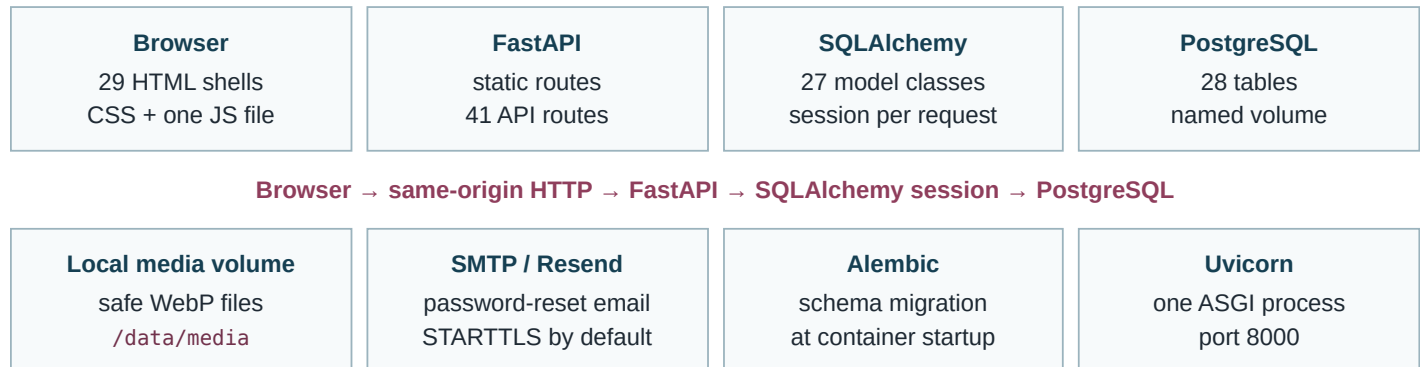
- Account registration and login, bcrypt-based password storage, JWT session cookies, bearer-token support, logout, password reset, session revocation, profile settings, export, and account anonymization.
- Published surgeon directory and search; canonical surgeon routes; shared data-driven Info, Reviews, Talk, Edit, and History shells.
- Sourced surgeon submissions with editor approval, administrator removal, and editor restoration.
- Edit proposal submission and trusted-editor approval into a new immutable surgeon revision.
- Review submission with controlled procedures/techniques, optional uploads, structured ratings at the API level, and pending moderation state.
- Talk-topic creation, private conversations, message replies, reports, account controls, and administrator role/ban/review-removal controls.
- Container startup, migration application, optional demo seeding, health checks, PostgreSQL persistence, local media persistence, and an operations-oriented backup script.

Core architectural judgment

The central relational design is sound for the stated product. A surgeon is the stable hub; article revisions, reviews, practices, procedures, discussions, and proposals attach to that hub without collapsing into one document. That preserves the most important product boundary: verifiable profile facts are not editable through patient reviews, and subjective experiences do not become factual biography text.

2. System architecture

High-level component map



Typical public request

1. A visitor requests `/surgeons/mara-voss`.
2. `app.main` returns the shared `frontend/pages/index.html` file. The slug stays in the URL.
3. `frontend/assets/js/script.js` parses the path and calls `GET /api/v1/surgeons/mara-voss`.
4. `app.api` obtains a SQLAlchemy session through `get_db()`, selects the published surgeon, and passes it to `surgeon_detail()`.
5. `app.serializers` combines the stable surgeon row, current immutable revision snapshot, procedure links, country display name, and published review count.
6. The browser replaces placeholder shell content, rewrites navigation links to preserve the slug, and removes the loading state.

Typical authenticated write

1. The browser sends JSON or multipart form data with the signed `HttpOnly` cookie automatically attached.
2. `current_user()` checks a bearer token first, then the `opensurgery_session` cookie.
3. The HS256 JWT is decoded using `SECRET_KEY`. The user is loaded from the database and checked for active/deleted status and matching `session_version`.
4. Pydantic validates the request schema; route-specific authorization checks the current database role where required.
5. The route writes domain records and, for sensitive changes, an `audit_events` row in the same commit.
6. The browser displays the returned state. Many content writes intentionally become `pending` rather than public.

Static and dynamic boundary

The HTML pages contain complete layouts and often contain fictional placeholder content. JavaScript replaces that content on dynamic pages. Surgeon-specific shells use `aria-busy="true"` and CSS hides their placeholder content until hydration finishes, preventing the Mara Voss example from flashing when another surgeon is requested. Policy and general navigation pages are static. There is no server-side templating layer; the server returns files and JSON separately.

3. Repository and file map

Root files

File	Purpose and connections
<code>README.md</code>	Primary setup guide. Documents Compose and host development, Resend SMTP, demo seeding, test invocation, and links to deeper product/backend documents.
<code>requirements.txt</code>	Pins FastAPI, Uvicorn, SQLAlchemy, Alembic, psycpg, pydantic-settings, bcrypt, PyJWT, multipart parsing, email validation, Pillow, and pycountry.
<code>Dockerfile</code>	Builds a Python 3.12 slim image, installs dependencies, creates UID/GID 10001, copies app/frontend/migrations, runs as non-root, exposes 8000, and uses the entrypoint.
<code>docker-compose.yml</code>	Defines PostgreSQL 17 Alpine and the app. Wires credentials, settings, named database/media volumes, dependency health, host ports, and restart policies.
<code>docker-entrypoint.sh</code>	Runs Alembic upgrades, optionally runs idempotent demo seed data, then <code>execs</code> Uvicorn.
<code>alembic.ini</code>	Alembic script location, fallback database URL, and migration logging.
<code>pytest.ini</code>	Points pytest at <code>tests/</code> , puts the repo on the Python path, and enables quiet output.
<code>.env.example</code>	Deployment configuration template. The real ignored <code>.env</code> exists locally but was not read into this report.
<code>.gitignore</code> , <code>.dockerignore</code>	Exclude secrets, virtual environments, caches, local databases/media, tests/docs from the image, and partial files.

Backend package: `app/`

File	Role
<code>__init__.py</code>	Marks the directory as a Python package.
<code>main.py</code>	Creates the FastAPI app, includes the API router, mounts assets, and maps root, canonical surgeon routes, and individual HTML filenames.
<code>api.py</code>	The application service layer: 41 route handlers spanning auth, account, directory, revisions, reviews, discussions, messages, media, reports, and moderation.
<code>config.py</code>	Central typed settings object loaded from environment variables and <code>.env</code> .
<code>database.py</code>	Engine, declarative base, session factory, SQLite compatibility switch, and request-scoped session dependency.
<code>models.py</code>	All enums, 27 SQLAlchemy model classes, one direct association table, constraints, relationships, and composite indexes.
<code>schemas.py</code>	15 Pydantic request models and the shared 8–128 character password policy.
<code>serializers.py</code>	Builds public surgeon summaries/details and review responses while controlling what crosses the privacy boundary.
<code>security.py</code>	Password prehash/bcrypt logic, JWT creation, cookie/bearer authentication, and session-version invalidation.
<code>email.py</code>	Delivers password-reset outbox items over SMTP using a worker-safe session; records attempts and sent state.
<code>countries.py</code>	Builds an alphabetized ISO country list from pycountry and normalizes codes or exact names.
<code>seed.py</code>	Idempotently loads fictional users, surgeons, procedures, techniques, ratings, revisions, review, talk, proposal, practice, and message fixtures.

Frontend: `frontend/`

- `assets/css/styles.css` — 914 lines of tokens, base components, page-specific layouts, responsive breakpoints, hydration states, and the later OpenSurgery identity override.

- `assets/js/script.js` — 964 lines of navigation behavior, API helper, authentication UX, page detection, hydration, form submission, dialogs, filters, and administrative controls.
- `assets/data/history-export.json` — a static fictional prototype export for the Mara Voss history screen, not a live export endpoint.
- `pages/` — 29 hand-authored HTML pages. Dynamic pages use the shared script; no bundler, component framework, or template compiler is present.

Migrations, tests, docs, and operations

- `migrations/env.py` points Alembic at runtime settings and `Base.metadata.script.py.mako` is the migration template.
- `migrations/versions/` holds the five-step schema chain described later.
- `tests/test_backend.py` contains the full 17-test suite and uses an isolated SQLite database at `/tmp/opensurgery-pytest.db`.
- `docs/product.md` is the product specification and status narrative. `docs/backend.md` is the deployment and data-boundary guide. `docs/research-notes.md` contains informal discovery notes and legal/design research reminders.
- `docs/screenshots/` contains 11 reference PNGs for desktop/mobile surgeon info, edit, history, reviews, revision, and talk views.
- `scripts/backup.sh` backs up PostgreSQL, uploaded media, environment configuration, commit/state metadata, Compose output, and checksums.
- `scripts/deploy.sh` attempts a pre-deploy backup, fast-forward pull, image build, app-container replacement, frontend hash check, and health verification. It currently contains path defects detailed in the findings.

4. Frontend architecture and screens

Browser application model

The frontend is progressively hydrated static HTML. One global JavaScript file discovers the current page by URL, finds page-specific DOM hooks, fetches the required API resources, and replaces placeholder regions. The API helper always uses same-origin credentials and JSON unless the body is `FormData`. Errors use the API's `detail` field when present.

Authentication display has two layers: the secure server cookie determines real access; a small `localStorage` value only changes navigation labels and lets forms show early “log in” messages. Server routes remain authoritative, and administrator controls are inserted only after `GET /me` reports the role.

Screen inventory

Page	Purpose	Data/behavior	Status
<code>home.html</code>	Landing and discovery	Loads up to three surgeons from <code>/surgeons</code> .	wired
<code>directory.html</code>	Surgeon directory	Server-backed name/country/procedure filtering and dynamic result cards.	wired
<code>search.html</code>	Cross-content search	Server surgeon query plus client filtering of procedure records; tabs filter rendered results.	partial
<code>procedures.html</code>	Controlled procedure guide	Hydrates procedure descriptions and links into category sections.	wired
<code>index.html</code>	Canonical surgeon Info shell	Loads profile/revision snapshot, procedures, counts, links, article sections, and admin remove control.	wired
<code>reviews.html</code>	Reviews tab for a surgeon	Loads surgeon and paginated published reviews; “load more” uses offset/limit.	partial UI
<code>review.html</code>	Single review	Hydrates title, byline, narrative, surgeon links; admin can hide the review.	partial fields
<code>talk.html</code>	Editorial discussion	Loads published topics/comments and lets signed-in users create a topic with its first comment.	partial
<code>edit.html</code>	Edit proposal form	Loads current surgeon and procedures, then submits article/profile snapshot and optional source.	wired
<code>history.html</code>	Published revision list	Loads immutable history; filters client-side; links to historical snapshot.	wired
<code>revision.html</code>	Historical version	Loads one revision and displays its article body and metadata.	partial snapshot
<code>write-review.html</code>	Review submission	Loads surgeon/procedure/technique choices, uploads images, submits pending review.	partial fields
<code>add-surgeon.html</code>	New profile submission	Loads countries and procedures; requires source; creates pending surgeon record.	wired
<code>pending.html</code>	Proposal queue	Public proposal summaries; authenticated editors additionally load and approve pending surgeons.	partial moderation
<code>practice.html</code>	Practice detail	Hardcoded Northbank slug, then loads locations and linked surgeons.	single record
<code>profile.html</code>	Public member profile	Hydrates name/bio, sends message, and injects admin user controls.	partial activity
<code>account.html</code>	Account dashboard	Uses cached display name and links; sign-out is wired.	mostly static
<code>settings.html</code>	Profile/contact/notification/privacy settings	Loads and patches <code>/me</code> ; live export and account deletion.	wired
<code>messages.html</code>	Private messages	Loads conversations/messages and posts replies.	wired

Page	Purpose	Data/behavior	Status
<code>create-account.html</code>	Registration	Posts registration and stores cosmetic local identity.	wired
<code>login.html</code>	Login	Posts credentials and stores cosmetic local identity.	wired
<code>forgot-password.html</code>	Reset request	Queues reset token/email while keeping enumeration-safe response text.	wired
<code>reset-password.html</code>	Set new password	Reads token from URL fragment, removes it from address bar, confirms reset.	wired
<code>guidelines.html</code>	Community rules	Static safety and conduct text.	static
<code>policies.html</code>	Editorial/review/advertising policies	Static prototype policy text.	draft
<code>privacy.html</code>	Privacy explanation	Static prototype text; one claim is now outdated.	stale
<code>terms.html</code>	Terms	Explicitly says it is product requirements rather than launch-ready legal terms.	draft
<code>surgeon.html</code>	Legacy/simple dynamic profile	Query-string hydration path separate from canonical <code>index.html</code> .	redundant
<code>talk-history.html</code>	Talk archive mockup	Static; no current API hydration.	prototype only

Styling system

The CSS first defines generic ink, line, paper, blue, and pink tokens, then later adds an explicit OpenSurgery identity layer with navy, deep navy, blue wash, pink wash, canvas, and card-border tokens. The later rules intentionally move the interface away from Wikimedia trade dress. Major layout patterns include article/sidebar grids, responsive directory and dashboard cards, review detail/form layouts, message split panes, policy layouts, history rows, editor sidebars, and a generated navigation drawer.

Responsive behavior is implemented primarily at 1050, 800/780/760, and 560 pixel thresholds. Desktop sidebars collapse or move below content, directory/forms become single-column, the global search wraps below the header, messages become a vertical layout, and revision sidebars collapse. Accessibility helpers include skip-link styling, screen-reader-only labels, visible focus outlines, semantic landmarks in the pages, `aria-busy` hydration states, and live status regions. There is no automated accessibility test in the repository.

JavaScript responsibilities

- Global navigation drawer, search redirects, language/more menus, copy-link handling, photo reveal, contents collapse, and prototype buttons.
- Shared `apiRequest()` wrapper with same-origin cookies, JSON parsing, and error normalization.
- Canonical surgeon URL parsing and link rewriting so Info/Reviews/Talk/Edit/History preserve the selected slug.
- Page-specific data loading for home, directory, search, surgeon, reviews, review detail, history, revision, talk, procedures, practice, profile, pending queue, messages, settings, and editor forms.
- Auth forms, password-reset forms, uploads, review creation, surgeon creation, edit proposals, conversations, account actions, and administrator actions.
- Prototype-only behavior remains for save draft, preview, history restore buttons, some filters, watch/subscribe buttons, and static talk history.

5. Backend module breakdown

app.main: ASGI application and file routing

The app metadata comes from `settings.site_name`; interactive docs are at `/api/docs` and OpenAPI JSON at `/api/openapi.json`. The API router is registered first, then `/assets` is mounted. Root returns `home.html`. Canonical surgeon URLs return shared shells for Info, Reviews, Talk, Edit, and History. A final one-segment catch-all only serves filenames already present in `frontend/pages`, preventing arbitrary path reads.

app.config: environment contract

Pydantic Settings reads environment variables and a local `.env`, ignoring unknown values. Defaults support local development but are not all production-safe. Settings cover site name, database URL, signing secret, token lifetime, cookie security, media path, public URL, SMTP host/port/credentials/sender, TLS mode, and timeout.

app.database: persistence boundary

A single engine uses `pool_pre_ping=True`. SQLite gets `check_same_thread=False` for tests. `SessionLocal` uses `expire_on_commit=False`, allowing route code to return committed object attributes without automatic refresh. `get_db()` yields and closes one session per dependency use. Most route handlers explicitly commit; there is no general automatic rollback middleware.

app.security: passwords and sessions

Passwords are SHA-256 hashed, base64 encoded, and then bcrypt hashed. The stored format is prefixed `bcrypt-sha256$`; verification retains compatibility with older direct bcrypt hashes. This avoids bcrypt's 72-byte truncation. JWTs use HS256, contain user ID, role, session version, and expiration, and default to 30 minutes. Authentication reloads the user, so role and active status come from current database state rather than trusting the token's role claim.

app.schemas: request validation

Fifteen request models constrain identity formats, string sizes, URLs, country/currency codes, rating payloads, media lists, role values, lifecycle statuses, report target types, and password length. These schemas are the first validation layer; route handlers add cross-record checks such as technique/procedure compatibility, media ownership, valid country names, and role authorization.

app.api: service logic

The API module is intentionally direct: route handlers query models, enforce rules, construct new records, and commit. There is no separate repository/service/domain layer. That keeps a small project understandable, though the 518-line file now mixes authentication, moderation, directory, media processing, messaging, and email queuing. Two small guards separate editor-level and administrator-level actions.

app.serializers: public representation

`surgeon_summary()` calculates published review count and linked procedures. `surgeon_detail()` adds the current revision's article body and JSON snapshot. `review_data()` resolves related surgeon/user/procedure/technique records, approved photos, rating dimensions, and the long-term update status. Email, password hashes, moderator-private data, and message contents do not enter public serializer outputs.

app.email: reset delivery

Password-reset requests create a database outbox record and a FastAPI background task. The task opens a fresh SQLAlchemy session, builds a plain-text email, uses STARTTLS or SMTP SSL, authenticates to the configured relay, and uses a Resend idempotency header. Failed attempts increment the counter and remain pending; success marks the row sent and removes the reset URL from the payload. No recurring retry worker exists, and the module does not deliver `new_message` outbox items.

app.seed: fictional fixture world

The seed exits if any surgeon already exists. Otherwise it creates five users (one admin), four fictional surgeons, seven procedures, three chest techniques, three rating dimensions, one practice/location, nine total revisions, one published review with three ratings, one talk topic with two comments, one pending proposal, and one two-person conversation. Every fixture email uses `example.com`; every fixture account uses `prototype-password`.

6. Complete API catalog

All JSON routes use the `/api/v1` prefix. “Auth” means any active signed-in user. “Editor” means trusted editor, moderator, or administrator. “Admin” means administrator only.

Health, authentication, and account

Method and path	Access	Behavior
GET <code>/health</code>	Public	Executes <code>SELECT 1</code> ; returns status ok.
POST <code>/auth/register</code>	Public	Creates unique email/display name, hashes password, sets strict HttpOnly session cookie.
POST <code>/auth/login</code>	Public	Case-insensitive email/name login; rejects inactive/deleted users; sets cookie.
POST <code>/auth/logout</code>	Public	Deletes the session cookie.
POST <code>/auth/password-reset</code>	Public	Returns the same accepted response for all addresses; stores hashed one-hour token and queues email.
POST <code>/auth/password-reset/confirm</code>	Public + token	Locks valid token, changes password, increments session version, consumes token, removes siblings, audits.
GET <code>/me</code>	Auth	Returns private account identity, role, profile, and preferences.
PATCH <code>/me</code>	Auth	Updates optional settings fields and records an audit event.
GET <code>/me/export</code>	Auth	Exports account data plus the user's review and proposal summaries.
DELETE <code>/me</code>	Auth	Anonymizes identity, clears profile/contact permission, deactivates account, records deletion time and audit.

Administrator accounts and review controls

Method and path	Access	Behavior
GET <code>/admin/users/{display_name}</code>	Admin	Returns target ID, role, active/ban state and reason.
PATCH <code>/admin/users/{display_name}/role</code>	Admin	Changes role; prevents demoting the last active administrator; audits private transition.
POST <code>/admin/users/{display_name}/ban</code>	Admin	Deactivates target, prevents self-ban, stores actor/time/reason, audits.
POST <code>/admin/reviews/{slug}/remove</code>	Admin	Changes review moderation state to hidden and records private reason.

Surgeons, lifecycle, revisions, and proposals

Method and path	Access	Behavior
GET <code>/surgeons</code>	Public	Lists published surgeons; filters name/location, normalized country, and linked procedure.
POST <code>/surgeons</code>	Auth	Creates unique slug, pending surgeon, initial revision, required source, procedure links, and audit event.
GET <code>/moderation/surgeons</code>	Editor	Lists pending surgeon submissions with current article, sources, and submitter.
POST <code>/moderation/surgeons/{id}/approve</code>	Editor	Publishes a pending surgeon and audits the approval.
POST <code>/moderation/surgeons/{slug}/remove</code>	Admin	Marks removed or retired, unpublishes, stores reason/actor/time, audits.
POST <code>/moderation/surgeons/{slug}/restore</code>	Editor	Restores removed/retired surgeon to published and clears removal fields.
GET <code>/surgeons/{slug}</code>	Public	Returns only published surgeon detail with current revision snapshot.

Method and path	Access	Behavior
GET /surgeons/{slug}/history	Public	Returns all published revision rows newest first.
GET /surgeons/{slug}/revisions/{number}	Public	Returns one immutable article/profile snapshot and metadata.
POST /surgeons/{slug}/proposals	Auth	Creates pending proposed article/profile snapshot and optional source.
GET /proposals	Public	Returns pending proposal summaries, surgeon summary, author, and timestamp.
POST /moderation/proposals/{id}/approve	Editor	Creates next reviewed revision, updates current pointer/structured surgeon fields/procedure links, copies sources, audits.

Reviews, reference data, profiles, and discussion

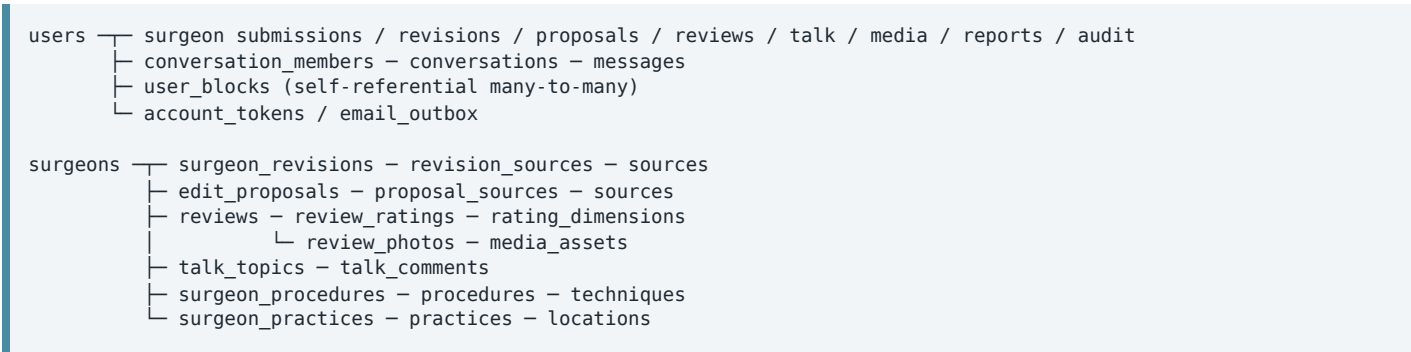
Method and path	Access	Behavior
GET /surgeons/{slug}/reviews	Public	Paginated published reviews for a published surgeon; limit clamped to 1–100.
GET /reviews/{slug}	Public	Published review only when its surgeon is also published.
POST /reviews	Auth	Validates surgeon/procedure/technique/ratings/media; creates pending review and photo links.
POST /media	Auth	Accepts up to 10 MiB JPEG/PNG/WebP; decodes, resizes to 4096 bounds, RGB converts, WebP re-encodes, hashes, stores.
GET /procedures	Public	Returns procedure catalog grouped by category with techniques.
GET /countries	Public	Returns pycountry ISO code/name pairs.
GET /practices/{slug}	Public	Returns practice public data, locations, and published linked surgeons.
GET /users/{display_name}	Public	Returns active user's pseudonymous public profile and message permission.
GET /surgeons/{slug}/talk	Public	Returns published topics and published comments in time order.
POST /surgeons/{slug}/talk	Auth	Creates a published topic and its first published comment.

Messaging, reports, and conversations

Method and path	Access	Behavior
POST /messages	Auth	Creates a new two-person conversation if recipient accepts messages; queues notification outbox row.
GET /conversations	Auth	Lists memberships with other member, latest non-deleted message, and recency sorting.
GET /conversations/{id}/messages	Member	Checks membership, returns non-deleted messages, updates last-read time.
POST /conversations/{id}/messages	Member	Checks membership, writes reply, queues notification rows for opted-in other members.
POST /reports	Auth	Validates UUID syntax, stores report target/reason/details, and audits report creation.

7. Database model and relationships

The schema contains 27 mapped classes plus the `surgeon_procedures` association table: 28 database tables in total. UUIDs are used for entity keys. Slugs provide stable human-facing lookup. Time-bearing records use timezone-aware timestamps.



Identity, accounts, and operations

Table	Key fields	Purpose and links
users	email, display_name, password_hash, session_version, role, preferences, active/ban/delete fields	Account identity and public profile. Self-FKs retain who banned a user. Deletion is application-level anonymization.
account_tokens	user_id, purpose, token_hash, expires_at, used_at	Single-use hashed reset tokens; cascade on user deletion at database level.
email_outbox	recipient_user_id, template, payload_json, state, attempts, next_attempt_at, sent_at	Durable queue state for password reset and message notifications.
audit_events	actor_id, action, target_type/id, public_detail, private_detail	Permanent action trail with explicit public/private detail split.
reports	reporter_id, target_type/id, reason, details, state	Generic abuse/content reports. Target integrity is polymorphic and enforced in application logic only.
user_blocks	blocker_id + blocked_id composite key	Represents block relationships; currently no API uses it.

Surgeon facts, practices, and revisions

Table	Key fields	Purpose and links
surgeons	slug, name, specialty, location, site/contact, portrait, published/lifecycle, submitter/removal fields, current_revision_id	Stable identity and current structured index fields. Points to the active immutable revision.
surgeon_revisions	surgeon_id, revision_number, parent, author, article_body, snapshot_json, summary, change_type, kind, published_at	Immutable public history. Unique surgeon + revision number. Kinds are normal, reviewed, rollback.
sources	url, title, publisher, archive, accessed date, supported claim, submitter	Reusable citation metadata.
revision_sources	revision_id + source_id	Many-to-many citations for published revisions.
edit_proposals	surgeon/base revision/author, proposed article/snapshot, summary, moderation state, submission/decision fields	Review queue separate from published history.
proposal_sources	proposal_id + source_id	Sources travel with a proposal and are copied to the approved revision.
practices	slug, name, site, public email/phone	Practice identity independent of a surgeon.
locations	practice_id, city/region/country, address/accessibility, effective dates	Practice locations over time.
surgeon_practices	surgeon_id + practice_id, effective dates, primary flag	Many-to-many surgeon/practice membership.

Controlled clinical taxonomy

Table	Key fields	Purpose and links
procedures	slug, unique name, category, description, aliases	Controlled top-level procedure records.
techniques	procedure_id, slug, name, description, aliases	Controlled technique records nested under one procedure; unique name within procedure.
surgeon_procedures	surgeon_id + procedure_id, effective dates, citation_id	Many-to-many offerings with optional temporal and source metadata.

Reviews and media

Table	Key fields	Purpose and links
reviews	slug, surgeon/reviewer/procedure/technique, surgery date precision, location, cost, wait time, narrative, complications/revision details, state/timestamps	First-person experience. Surgeon/procedure deletion is restricted; reviewer deletion preserves the review with null attribution.
rating_dimensions	slug, label, description, active	Controlled rating categories.
review_ratings	review_id + dimension_id, value	One value per review/dimension. Range 1–5 is enforced in route logic rather than a database check.
media_assets	owner, original/safe key, type, size/dimensions, SHA-256, processing state	Metadata for stored media. Current implementation uses the same key for original and safe output because only the re-encoded output is retained.
review_photos	review/media, capture date precision, caption, visibility, warning, long-term designation, approved_at	Per-review media semantics and moderation. One media asset can belong to only one review photo.

Discussion and private communication

Table	Key fields	Purpose and links
talk_topics	surgeon_id, slug, title, author, moderation state	Article-specific editorial thread; slug unique per surgeon.
talk_comments	topic, author, parent comment, body, version, state, edit timestamps	Nested discussion records. The schema anticipates versions, but no comment-edit endpoint exists.
conversations	id, created_at	Container for private messages.
conversation_members	conversation_id + user_id, joined_at, last_read_at	Membership and read state.
messages	conversation, sender, body, created_at, deleted_at	Private messages with soft-delete field; notification emails intentionally omit body content.

Migration history

1. [645cd7ed22d8](#) — initial relational schema with all major product domains.
2. [7c4d2a90b816](#) — surgeon submission/removal lifecycle fields and backfill of published records.
3. [9b61e8f3a4c2](#) — user ban actor/time/reason fields.
4. [c82d4e71f9a0](#) — removes free-form surgeon aliases.
5. [2d4b2c91a173](#) — adds `session_version` for global session revocation after password reset.

8. End-to-end workflows

Registration and login

```
HTML form → POST /auth/register or /auth/login → validate identity/password  
→ create/load User → create signed JWT → HttpOnly SameSite=Strict cookie  
→ browser caches display name only → subsequent API calls authenticate from cookie
```

Registration enforces a 3–30 character alphanumeric/underscore/hyphen display name and validated email. Duplicate email/name becomes HTTP 409. Login compares email or display name case-insensitively. Cookie security depends on `COOKIE_SECURE`, false by default and configurable for TLS deployments.

Password reset

```
email request → generic 202 response → random 48-byte URL-safe token  
→ SHA-256 digest stored in account_tokens → raw token only in URL fragment  
→ outbox row + background SMTP task → user submits new password  
→ row lock + expiry/single-use checks → bcrypt-sha256 password  
→ session_version++ → all old sessions rejected
```

The URL fragment design keeps the reset token out of normal HTTP request lines and referrer headers. The browser removes the fragment from the address bar immediately after reading it. The SMTP delivery uses an idempotency key derived from the outbox UUID.

New surgeon lifecycle

```
member submits sourced record → surgeons(pending, unpublished)  
→ revision #1 + source + procedure links + audit  
→ editor queue → approval → published  
→ admin may remove/retire → hidden from public queries  
→ editor may restore → published again
```

The initial revision exists before approval, but public detail endpoints filter on `is_published`. Removal keeps history and relationships. The lifecycle string and the legacy `is_published` flag are maintained together by route logic.

Edit proposal and publication

```
browser loads current surgeon + procedures → member edits article and structured profile  
→ pending edit_proposal references current revision → optional source rows  
→ editor approval calculates max revision + 1  
→ immutable surgeon_revision created → current_revision_id updated  
→ selected structured fields and procedure links synchronized → sources copied → audit
```

This is the strongest workflow in the project: proposed data is separated from authoritative data, approval creates history rather than overwriting it, and a parent pointer preserves lineage. The current API has approval but no proposal-rejection, rollback, draft, or conflict-detection endpoint.

Review and photo submission

```
browser loads surgeon/procedure/technique catalogs  
→ optional images POST /media one-by-one  
→ decoded, resized, RGB-converted, WebP-reencoded, stored and recorded ready  
→ POST /reviews validates relationships, ratings and media ownership  
→ pending Review + ReviewRating + ReviewPhoto records
```

Review publication stops at pending because there is no approval endpoint. New photo links have no capture date, visibility choice, caption, warning, or `approved_at`. The public serializer only considers approved photos, so user-submitted long-term images cannot currently make the public long-term status become “included.”

Long-term update calculation

- No designated approved photo: “Missing 1-year update.”
- A designated approved photo with missing surgery or capture date: “Date unavailable.”
- Exact surgery date and a designated capture date: included only if at least 365 days later.
- Imprecise surgery date with a designated approved photo: trusts the designation and marks included.

Messaging

```
sender opens public profile → recipient must allow_messages  
→ new conversation + two memberships + first message  
→ message-notification outbox row if opted in  
→ membership-gated list/read/reply endpoints → last_read_at updated
```

Every initial message creates a new conversation, even for the same pair. The block table and `allow_new_accounts` preference are not consulted. Notification outbox rows for messages are created but never delivered by the existing email module.

Moderation and administration

Trusted editors can approve proposals and surgeon submissions and can restore removed surgeons. Administrators additionally inspect account state, change roles, ban accounts, remove reviews, and remove/retire surgeon profiles. The browser only creates admin controls after confirming the role from the server, and every endpoint repeats authorization server-side. Significant actions create audit events, though coverage is not complete for every possible state change.

9. Security and privacy model

Existing safeguards

- **Password handling:** centralized minimum/maximum length, SHA-256 prehash, bcrypt work factor, compatibility with older bcrypt hashes.
- **Sessions:** short-lived signed tokens, HttpOnly cookies, SameSite Strict, optional Secure flag, current-user database lookup, active/deleted checks, session-version invalidation.
- **Reset privacy:** generic response prevents account enumeration; raw token is never stored; one-hour expiration; row lock; one-time use; URL fragment transport.
- **Authorization:** authenticated/editor/admin requirements are enforced in route code. UI checks are cosmetic only.
- **Public data boundary:** serializers omit email, password hash, private moderator detail, and private messages.
- **Uploads:** size cap, format allowlist, full decode, resize, RGB conversion, WebP re-encode, generated UUID key, content hash, ownership validation on review attachment.
- **Container:** app process runs as a dedicated non-root user.
- **Audit:** account reset/settings/deletion, reports, surgeon lifecycle, proposal approval, role changes, bans, and review removal generate events.

Security limitations

- No application rate limiting for signup, login, password reset, messages, reports, edits, or uploads.
- No reverse proxy or TLS configuration is included. `COOKIE_SECURE=false` is the provided default.
- No CSRF token mechanism. SameSite Strict substantially reduces ordinary cross-site form attacks, but explicit CSRF protection would give stronger defense and clearer guarantees.
- Local media storage has no public delivery route, private authorization layer, malware scanning, quarantine, or isolated object-store origin.
- No Content Security Policy, security headers middleware, structured request logging, or dependency/security scanning configuration appears in the repository.
- The default development signing secret in `config.py` is unsafe if the app is launched outside Compose without replacing it. Compose correctly requires `SECRET_KEY`.
- Report target existence is not verified, and its polymorphic target has no database foreign key.
- Message blocking is modeled but not enforced.

Privacy behavior

Public profiles are pseudonymous and reveal only display name, bio, approximate region, message preference, and member-since date. Account export is authenticated. Account deletion anonymizes account identity while preserving contributions with a “Deleted member” fallback. Messages are membership-gated, and notification outbox payloads carry conversation IDs rather than message bodies.

Policy mismatch: `privacy.html` states that the prototype does not transmit or store form data. The current application does both. This text must be corrected before anyone relies on it.

10. Configuration, containers, deployment, and backup

Runtime settings

Variable	Use	Default / Compose behavior
<code>SITE_NAME</code>	FastAPI title and reset-email branding	<code>transdoc.wiki</code> ; not exposed in the example/Compose file.
<code>DATABASE_URL</code>	SQLAlchemy and Alembic connection	Local PostgreSQL default; Compose injects service URL.
<code>SECRET_KEY</code>	JWT signing	Development fallback in code; required by Compose.
<code>ACCESS_TOKEN_MINUTES</code>	JWT/cookie lifetime	30.
<code>COOKIE_SECURE</code>	Secure cookie attribute	false.
<code>MEDIA_ROOT</code>	Processed upload path	<code>var/media</code> locally; <code>/data/media</code> in Compose.
<code>PUBLIC_BASE_URL</code>	Password reset links	<code>http://127.0.0.1:8000</code> .
<code>SMTP_HOST/PORT/USERNAME</code>	Mail relay	Resend SMTP, 587, <code>resend</code> .
<code>SMTP_PASSWORD</code>	Relay credential/API key	Unset; reset delivery returns false without it.
<code>SMTP_FROM_ADDRESS</code>	From header	Optional; otherwise <code>no-reply@{site_name}</code> .
<code>SMTP_STARTTLS</code> , <code>SMTP_USE_SSL</code>	Transport security mode	true / false.
<code>SMTP_TIMEOUT_SECONDS</code>	SMTP socket timeout	10; not listed in example/Compose.
<code>SEED_DEMO_DATA</code>	Optional startup seed	false; must stay false for real environments.
<code>APP_PORT</code>	Host-side Compose port	8000.
<code>POSTGRES_PASSWORD</code>	Database credential	Example requires replacement; Compose otherwise falls back to <code>opensurgery</code> .

Container startup

1. PostgreSQL starts and must pass `pg_isready`.
2. The application entrypoint runs `alembic upgrade head`.
3. If explicitly enabled, `python -m app.seed` inserts fictional data once.
4. Uvicorn starts on `0.0.0.0:8000`.
5. The app health check opens `/api/v1/health`.

The PostgreSQL and media directories are named volumes, so ordinary application-container recreation does not erase them. PostgreSQL is also published on host port 5432, which is convenient for development but should be reviewed for production exposure.

Backup behavior

`backup.sh` writes timestamped directories under `/srv/backups/opensurgery`, temporarily stops the application if it is running, makes a custom-format `pg_dump`, validates the archive with `pg_restore --list`, tars the media volume, records Git commit/status and resolved Compose config, copies `.env` with mode 600, and calculates checksums. Its trap restarts the app.

Operational omissions include retention/pruning, encryption, off-host replication, restore automation/testing, monitoring/alerting, and a documented recovery objective.

Deployment behavior

The intended flow is backup → fast-forward pull → record commit → build app image → force-recreate only the app → compare the frontend JavaScript hash inside and outside the container → wait for healthy status. This is a practical verification sequence, but the script currently fails before reaching it because of hardcoded path errors listed in the findings.

11. Automated tests and validation

The suite was run during this audit and completed successfully: **17 passed in 4.28 seconds**. Tests point `DATABASE_URL` to a temporary SQLite file, drop and recreate all model metadata once, seed fictional data, and call route functions directly with real SQLAlchemy sessions.

What the 17 tests cover

1. Seeded relation counts and public surgeon/review/history/procedure queries.
2. Backend branding uses the central site-name setting.
3. Surgeon shells hide placeholder data and do not fall back to Mara Voss.
4. Canonical clean routes select correct HTML shells and reject invalid sections.
5. Shared eight-character password minimum.
6. Backward compatibility for tokens issued before session versioning.
7. Password verification, JWT creation, HttpOnly cookie, and public profile privacy.
8. Hashed, single-use reset tokens, outbox creation, new password, token reuse rejection, and session revocation.
9. SMTP password-reset delivery, content, branding, URL fragment, and idempotency header.
10. Settings, review/rating/photo, talk, message, and reply persistence.
11. Proposal approval creating a new revision and updating structured profile/procedures.
12. Optional source on edit proposals.
13. Static frontend/API contract presence and controlled procedure/technique data.
14. Complete country list and dynamic add-surgeon procedure controls.
15. Pending → approved → removed → restored surgeon lifecycle and role denial.
16. Administrator role, ban, login denial, review removal, and audit behavior.
17. Admin controls are absent from static HTML and created after server role confirmation.

What is not tested

- Real HTTP requests through FastAPI's ASGI stack, cookie round trips, or browser end-to-end behavior.
- Alembic migrations against PostgreSQL, including upgrade from each historical revision.
- Upload size/type/decompression edge cases, metadata stripping assertions, filesystem permissions, or media delivery.
- Concurrency: duplicate slugs, simultaneous revision approvals, repeated messages, reset races, or startup migration races.
- Rate limits, CSRF, security headers, accessibility, responsive rendering, or JavaScript unit behavior.
- Backup restore, deployment script execution, health failure rollback, or SMTP retry processing.
- Report moderation, block behavior, review/photo approval, proposal rejection, rollback, talk comment history, or notification delivery.

12. Implemented, partial, and planned behavior

Domain	Implemented	Partial or missing
Accounts	Register, login/logout, reset, settings, export, anonymizing deletion, roles, bans	Email verification, unban, role-management breadth, account recovery operations, age policy
Surgeon directory	Published listing, search/filter, detail, controlled procedures, practice link model	Scalable search/indexing, generic practice routing, richer effective-date display
Revisions	Immutable revisions, current pointer, history, snapshots, proposal approval, sources	Reject/draft/preview backend, rollback/undo endpoint, conflict detection, full snapshot rendering, public source detail
Reviews	Create pending review, read published review, ratings model, complications/cost schema, pagination	Review approval/rejection, edit/delete by author, UI ratings/cost/revision details, moderation queue
Photos	Safe re-encode, metadata-oriented model, ownership checks, long-term-status algorithm	Serving, visibility selection/enforcement, capture dates, captions/warnings, approval workflow, malware scanning/object storage
Talk	List topics/comments and create a topic with first comment	Reply endpoint, edit/version history, moderation queue, talk-history API
Messaging	New conversation, list/read/reply, read timestamps, notification outbox creation	Block enforcement, new-account restriction, existing-thread reuse, delete UI, delivered email worker
Reports/moderation	Create report, audit event, user ban, role change, profile/review removal	Report queue/resolution, appeals, unban, review/photo publish tools, complete moderator UI
Notifications/watch	Preferences stored; password-reset SMTP works	Watch model, watched-page notifications, new-message delivery, retry worker
Operations	Compose, health checks, migrations, non-root app, persistent volumes, backup concept	TLS/proxy, rate limits, monitoring, fixed deploy script, tested restore, secret manager, encrypted/offsite backup

13. Findings and prioritized risks

High: the public privacy statement is factually stale

`privacy.html` says the prototype does not transmit or store form data. Registration, settings, reviews, images, proposals, messages, reports, and other forms now persist data. This should be treated as a blocking launch issue because it misdescribes actual handling of sensitive information.

High: review and photo moderation cannot complete

New reviews default to pending and no endpoint publishes or rejects them. New review photos have `approved_at = NULL`; no route approves them. Public queries only show published reviews and approved photos, so normal submissions cannot enter the public product. The long-term photo feature is therefore structurally present but operationally unreachable.

High: deployment script path defects

`scripts/deploy.sh` begins with an unconditional `cd /home/ubuntu/docker/surgery-website/Bomboclat` before its robust `PROJECT_DIR` resolution. If that old path is absent, `set -e` is not yet active but the failed `cd` still leaves execution in an unknown directory. It then invokes `./home/ubuntu/docker/open-surgery/scripts/backup.sh`, which is a relative path that almost certainly does not exist. The intended pre-deploy backup and deployment sequence is not reliable as written.

High: production protections named in the product spec are not deployed

The repository has no reverse proxy/TLS, rate limiting, object storage, isolated media serving, malware scanning, recurring outbox worker, monitoring, or formal retention controls. These are especially relevant because the product handles accounts, private messages, surgery dates, and intimate medical imagery.

Medium: branding is split

Static pages and product docs say OpenSurgery; backend/API/email branding defaults to `transdoc.wiki`; cookie names, database names, volumes, backup paths, and many operational identifiers still say `opensurgery`. A user could see different names in the UI, API documentation, and password-reset email. `SITE_NAME` is not shown in `.env.example` or passed by Compose.

Medium: UI fields and database fields are out of sync

The review form asks for insurance but JavaScript discards it. The model supports cost, currency, included costs, wait time, complications detail, revision detail, ratings, photo capture dates, visibility, captions, and warnings; the current form sends almost none of these. Conversely, the profile page displays hardcoded contribution/activity counts rather than server data.

Medium: messaging preferences and blocks are not enforced

`allow_new_accounts` and `user_blocks` exist but sending and replying only check `allow_messages` at conversation creation. Replies ignore current recipient preference and blocks. Every new profile message opens a new conversation, producing duplicates.

Medium: queued message notifications never send

Message routes correctly enqueue `new_message` outbox records without message bodies. The email module only knows how to deliver `password_reset`. There is no worker for pending outbox rows or retry schedule, so message notification preferences currently have no external effect.

Medium: proposal approval has race and consistency edges

The next revision number is computed as `max + 1` without a locking strategy; concurrent approvals can violate the unique constraint. The proposal does not detect that its base revision became stale. Structured updates and procedure replacement are correct inside one transaction, but an HTTP error raised after some pending session changes relies on session disposal for rollback rather than an explicit error-handling pattern.

Medium: content-state rules are incomplete

Talk topics/comments are immediately published for any authenticated user. Pending proposals are publicly listed with author attribution. Reports can point to nonexistent UUIDs. There is no proposal rejection, review publication, report resolution, account unban, or appeal flow. These choices may be acceptable for a prototype but do not fulfill the documented moderation model.

Medium: media is stored but not consumable

The upload endpoint writes safe WebP files and records them, but no authorized route serves those files and review JSON omits photo URLs/metadata. The database has granular visibility, yet neither submission nor delivery enforces it. Local filesystem storage also conflicts with the documented production goal of isolated object storage.

Lower: search and filtering are prototype-scale

Surgeon name/city/region filtering uses substring matching; procedure filtering serializes each surgeon and filters in Python, causing repeated review-count and procedure queries. Search fetches the entire procedure catalog and filters it in the browser. This is clear and adequate for demo data but becomes inefficient as the directory grows.

Lower: duplicate and legacy UI paths increase maintenance

`surgeon.html` duplicates a simpler profile route while canonical surgeon pages use `index.html`. `talk-history.html` and the static history export are not connected to live data. Large shared CSS and JavaScript files contain both early prototype rules and later identity overrides, increasing the chance of regressions.

Lower: tests bypass important runtime layers

Direct route-function calls are fast and valuable but do not validate ASGI routing, serialization, cookies in a client, PostgreSQL-specific behavior, migrations, JavaScript, or actual browser screens. Passing tests therefore show core logic coherence, not production readiness.

14. Recommended next work

1. **Correct user-facing truth first.** Rewrite the privacy page to match current persistence; finalize operating entity/jurisdiction, terms, retention, photo access, moderation, appeals, and breach policies before public data collection.
2. **Complete the moderation loop.** Add review/photo approve/reject actions, proposal rejection, report queue/resolution, unban, and audited moderator interfaces. Make long-term photo approval and capture-date handling reachable.
3. **Finish media architecture.** Add quarantine/scanning, object storage, private/public variants, authorization-aware delivery, visibility controls, content warnings, capture metadata, cleanup, and deletion/retention behavior.
4. **Harden external exposure.** Put the app behind TLS, enable Secure cookies, add rate limits and security headers, protect state-changing cookie requests with explicit CSRF controls, and keep PostgreSQL off public interfaces.
5. **Fix operations before using deployment automation.** Repair deploy paths, validate backup invocation, add encrypted off-host retention, and run documented restore drills. Add monitoring for health, migration failure, SMTP backlog, disk use, and backup age.
6. **Build the outbox worker.** Dispatch both password-reset and message notifications, implement retry/backoff with `next_attempt_at`, cap attempts, and expose operational failure state without leaking private content.
7. **Enforce communication controls.** Apply blocks and new-account preferences on new messages and replies; reuse existing direct conversations; add block/unblock APIs and UI.
8. **Align forms with the model.** Decide which review fields belong in the MVP, then either collect/display them or remove misleading controls. Insurance needs a real schema decision because it currently exists only in HTML.
9. **Consolidate branding and shells.** Choose the public name, expose it consistently to HTML and backend, update examples and operational identifiers where appropriate, retire the duplicate surgeon page, and connect or remove prototype-only pages.
10. **Expand verification proportionally.** Add FastAPI HTTP integration tests, PostgreSQL migration tests, upload security tests, browser tests for core flows, accessibility checks, and deploy/restore smoke tests. Add concurrency coverage around revision numbering and reset consumption.
11. **Optimize only after real usage.** Once correctness is complete, remove N+1 directory queries with eager/aggregate queries and measure PostgreSQL search before adding a separate search service.

15. Appendices

A. Dependency map

Dependency	Version	Used for
FastAPI	0.116.1	ASGI app, routing, dependency injection, uploads, background tasks, static files
Uvicorn standard	0.35.0	Production/development ASGI server
SQLAlchemy	2.0.43	ORM, schema metadata, SQL expression queries, sessions
Alembic	1.16.5	Database schema migration
psycopg binary	3.2.10	PostgreSQL driver
pydantic-settings	2.10.1	Typed environment and <code>.env</code> configuration
bcrypt	4.3.0	Password hashing
PyJWT	2.10.1	Signed session/access tokens
python-multipart	0.0.20	Multipart image upload parsing
email-validator	2.2.0	Pydantic email validation
Pillow	11.3.0	Image decode, resize, conversion, and WebP output
pycountry	24.6.1	ISO country catalog and normalization

B. Complete tracked-file inventory

Root/config	add-surgeon.html
.dockerignore	create-account.html
.env.example	directory.html
.gitignore	edit.html
Dockerfile	forgot-password.html
README.md	guidelines.html
alembic.ini	history.html
docker-compose.yml	home.html
docker-entrypoint.sh	index.html
pytest.ini	login.html
requirements.txt	messages.html
Backend	pending.html
app/__init__.py	policies.html
app/api.py	practice.html
app/config.py	privacy.html
app/countries.py	procedures.html
app/database.py	profile.html
app/email.py	reset-password.html
app/main.py	review.html
app/models.py	reviews.html
app/schemas.py	revision.html
app/security.py	search.html
app/seed.py	settings.html
app/serializers.py	surgeon.html
Frontend assets	talk-history.html
frontend/assets/css/styles.css	talk.html
frontend/assets/js/script.js	terms.html
frontend/assets/data/history-export.json	write-review.html
Frontend pages	Migrations
account.html	migrations/env.py
	migrations/script.py.mako

migrations/versions/645cd7ed22d8_initial_relational_schemas.py
migrations/versions/7c4d2a90b816_add_surgeon_lifecycle.py
migrations/versions/9b61e8f3a4c2_add_user_ban_audit_fields.py
migrations/versions/c82d4e71f9a0_remove_surgeon_aliases.py
migrations/versions/2d4b2c91a173_add_user_session_version.py
Tests/ops/docs
tests/test_backend.py

scripts/backup.sh
scripts/deploy.sh
scripts/.swp
docs/backend.md
docs/product.md
docs/research-notes.md
11 PNG files under docs/screenshots/

C. Terminology

Term	Meaning in this project
Surgeon identity	The stable surgeons row that connects every public and editorial domain.
Revision	An immutable published article body plus structured JSON snapshot.
Proposal	A pending replacement article/profile snapshot awaiting editor action.
Moderation state	draft, pending, published, rejected, or hidden.
Lifecycle status	Surgeon-level pending, published, removed, or retired string state.
Long-term update	An approved designated photo whose reported timing satisfies the product rule; not an authenticity or outcome judgment.
Editor	Any trusted editor, moderator, or administrator for editor-gated routes.
Outbox	Database record describing an email to deliver, its attempts, and state.
Hydration	Browser JavaScript replacing a static HTML shell with API data.

D. Audit method and limits

This report was produced by inspecting every tracked source/configuration path, reading all backend modules, routes, schemas, models, migrations, deployment scripts, product documentation, frontend page hooks, JavaScript integration logic, CSS structure, and the complete test suite. The test suite was executed. Git history and worktree state were checked. Private environment values were not copied into the report. The screenshots were inventoried as reference artifacts; conclusions about behavior come from the source and tests rather than visual inspection alone.

The report describes revision **7263815** as it existed on September 20, 2026. It does not assert that external services, DNS, SMTP credentials, deployed hosts, backups, or production databases are currently configured or healthy.